

More Effective Coroutines Pro

More Effective Coroutines Pro runs on the exact same super fast and super optimized core that the free version runs on, and also adds a host of additional methods, most of which can't be found in Unity's coroutines or anywhere else. MEC Pro can be found on the Unity asset store [here](#).

This document covers Pro-only features. For setup, migration from Unity coroutines, and core functionality, see the Quick Start Guide.

Additional Segments

MEC Pro adds several timing segments beyond the Update, FixedUpdate, LateUpdate, and SlowUpdate segments available in MEC Free.

Realtime Update

RealtimeUpdate is just like Update, but Timing.LocalTime and Timing.DeltaTime ignore Unity's timescale. This can be useful while controlling menus that happen while your game is paused (if you pause by setting the timescale to 0).

```
Timing.RunCoroutine(_AnimateMenu(), Segment.RealtimeUpdate);
```

End Of Frame

The EndOfFrame segment executes as the very last action before the frame is done, after all rendering has finished.

```
Timing.RunCoroutine(_CaptureScreenshot(), Segment.EndOfFrame);
```

Late Fixed Update

LateFixedUpdate executes when FixedUpdate happens, but after it has finished its work. This is useful for operations that need to read the results of physics calculations performed in FixedUpdate.

```
Timing.RunCoroutine(_ReadPhysicsResults(), Segment.LateFixedUpdate);
```

Manual Timeframe

The ManualTimeframe segment can be configured to execute and define its notion of time in custom ways, giving you full control over when coroutines in this segment tick.

Layers and Tags

The free version of MEC only has tags, but MEC Pro adds layers. Collectively tags and layers are called graffiti because they are both just ways to identify a particular instance of a coroutine. The only real difference between a layer and a tag is that a layer is an integer and a tag is a string, but in MEC Pro you have the option of supplying one, the other, or both. Once you supply graffiti for an instance, you can pause/resume, kill, or use that graffiti for RunCoroutineSingleton.

In Unity every GameObject is assigned a unique number, which can be accessed by calling `gameObject.GetInstanceID()`. The instance id can change every time you run the application, but it is guaranteed that no other gameObject instance will be assigned the same id during a single run. So

even if you have a swarm of enemies that all have the same scripts attached to them you can still run them on a layer which is the instance id of the gameObject they were created on and then kill all coroutines attached to one particular enemy using:

```
Timing.KillCoroutines(enemy.gameObject.GetInstanceID());
```

If you had, say, an amnesia gun you could graffiti all your enemy's AI logic with both the instance id and a tag:

```
Timing.RunCoroutine(_EnemyAI(),
    gameObject.GetInstanceID(), "AI");

// Somewhere else in code:
void HitEnemyWithAmnesiaGun(EnemyController enemy)
{
    Timing.KillCoroutines(
        enemy.gameObject.GetInstanceID(), "AI");
}
```

KillCoroutines only kills the instances that match all the graffiti that you pass in, so if you coded it right the above code would make the enemy stand there and do nothing, but would also leave any other coroutines that might be running on that particular character untouched.

Unity's coroutines will always cancel all of the coroutines that are associated with a GameObject whenever you disable that object. Sometimes you would rather just pause a coroutine while it's disabled and then resume it as soon as the object is re-enabled. That sort of pattern is really easy to set up in MEC like this:

```
void Start ()
{
    Timing.RunCoroutine(_MoveUpAndDown(),
        gameObject.GetInstanceID());
}

void OnEnable()
{
    Timing.ResumeCoroutines(gameObject.GetInstanceID());
}

void OnDisable()
{
    Timing.PauseCoroutines(gameObject.GetInstanceID());
}
```

RunCoroutineSingleton

You often want to run a coroutine, but you don't want to run multiple copies of that coroutine. One common use case is to have a button that visibly pops onto the screen and back out as it is enabled and disabled. You might write a coroutine that expands the transform of that button every frame in a very pleasing lerp effect, but find that if the user clicks back and forth very quickly then more than one coroutine can be started at the same time in a very unpleasing way.

RunCoroutineSingleton allows you to fix that. The normal way to use it is to apply a unique tag to every coroutine in the set (for this example that would be every coroutine that moves that button):

```
Timing.RunCoroutineSingleton(
    _ButtonPopIn(button.transform),
    "HealthButtonMovementSet",
```

```
SingletonBehavior.Overwrite);
```

Tags do result in a memory alloc, so if you are trying to be as efficient as possible with your GC allocs then you might want to use a handle to define your singleton:

```
class Foo
{
    CoroutineHandle handle;

    void Start()
    {
        handle = Timing.RunCoroutineSingleton(
            _MoveThatThing(), handle,
            SingletonBehavior.Overwrite);
    }
}
```

The third parameter that you pass in is an enum that determines how any conflicts that are found will be handled. **Abort** will fail to run the current coroutine if one is already defined that matches your parameters. **Overwrite** will kill any matching coroutines and then run yours. **Wait** will automatically keep your coroutine paused until all the matches have finished and then it will execute.

One thing that you can do with the Wait behavior is to set up a sequence of coroutines to run one after the other by giving them all the same tag and setting them to Wait. Each coroutine will only wait for the coroutines that exist with that tag at the moment it's created; later additions with the same tag will be ignored.

Below are some examples of usage:

```
Timing.RunCoroutineSingleton(
    _ButtonPopIn(button.transform),
    "HealthButtonMovementSet",
    SingletonBehavior.Overwrite);

Timing.RunCoroutineSingleton(
    _ButtonPopIn(button.transform),
    "HealthButtonMovementSet",
    SingletonBehavior.Wait);

CoroutineHandle handle;
handle = Timing.RunCoroutineSingleton(
    _ButtonPopIn(button.transform), handle);

handle = Timing.RunCoroutineSingleton(
    _ButtonPopIn(button.transform),
    Segment.FixedUpdate, handle);
```

Controlling Coroutines by Handle

In MEC Free, **CoroutineHandle** is primarily used with **KillCoroutines**, **PauseCoroutines**, **WaitUntilDone**, and **IsRunning**. In MEC Pro, the handle itself exposes properties and methods for richer control.

Tag and Layer

You can get or set the associated graffiti directly on a handle:

```
CoroutineHandle handle = Timing.RunCoroutine(_Foo());
```

```
string oldTag = handle.Tag;
handle.Tag = "newTag";

if(handle.Layer == null) // No layer is assigned
    handle.Layer = gameObject.GetInstanceID();
```

NOTE: Layer is a nullable int (`int?`) so it may require casting into a regular int in your code. You should check whether the value is null when you query either tags or layers — a null value means that there is no tag or layer assigned.

Segment

You can retrieve or change the timing segment that a coroutine is running in:

```
handle.Segment = Segment.SlowUpdate;
```

IsRunning, IsAliveAndPaused, and IsValid

IsRunning returns true until the coroutine terminates, and then it returns false. Paused and/or held coroutines are considered to be running. Setting **IsRunning** to false will kill the coroutine.

IsAliveAndPaused returns true if the coroutine is paused. Setting this value will pause or resume the coroutine.

IsValid returns true if the coroutine handle has ever pointed to a valid coroutine (regardless of whether that coroutine is currently running).

OnDestroy

You can register a callback that fires when a coroutine ends, whether it finishes naturally, throws an exception, or is killed:

```
CoroutineHandle handle = Timing.RunCoroutine(_LoadAssets());
handle.OnDestroy(() => Debug.Log("Loading complete ")).
```

NOTE: It is generally a bad idea to use `OnDestroy` on any coroutine that you would use `CancelWith` on, because that will typically lead to exceptions when the `GameObject` is destroyed.

Linking Coroutine Handles

Handles can be linked to one another using the **LinkCoroutines** function. This means that the "master" coroutine will send a kill command to the "slave" coroutine when it ends (either from a kill command or just by ending the function). It will also copy any pause or resume commands (but not any calls to `WaitForSeconds`).

The reason you would want to link coroutines is so you can start two or more coroutines independently but treat the whole group like it's a single coroutine with the same scope as the master coroutine. A simple example could be if you have a loading progress bar with a coroutine that updates the progress bar's position. Let's say that loading can sometimes get stuck and you want to create a second coroutine to show "sorry for the delay" when it stalls. Let's say there's also a cancel button which kills your first coroutine and a pause button which pauses it. You could turn your single reference into a list, but that starts getting messy fast. The simple way is to just link the two coroutines in MEC and then you can treat them like a single one. The slave will always be terminated or paused along with the master.

```
CoroutineHandle master = Timing.RunCoroutine(_LoadAssets());
CoroutineHandle slave = Timing.RunCoroutine(_ShowDelayWarning());
Timing.LinkCoroutines(master, slave);
```

Getting the Current Coroutine Handle

This is a useful convenience that allows you to retrieve your own handle from within a coroutine. You can use your own handle to set up links on the fly, change your own tags, coordinate static lists of different instances of the same coroutine, or all kinds of things.

```
private IEnumerator<float> _shout(float time, string text)
{
    yield return Timing.WaitForSeconds(time);
    CoroutineHandle myHandle = new CoroutineHandle();
    yield return Timing.GetMyHandle(x => myHandle = x);
    Debug.Log(myHandle.Tag + ": " + text);
}
```

NOTE: GetMyHandle is a special function that does not cause a one frame delay even though it's in a yield return statement.

Additional WaitUntilDone Overloads

MEC Pro adds shorthand overloads of WaitUntilDone that accept an **IEnumerator<float>** directly. These automatically start a new coroutine and hold the current one until it finishes:

```
// MEC Free requires the longhand version:
yield return Timing.WaitUntilDone(
    Timing.RunCoroutine(_OtherCoroutine()));

// MEC Pro shorthand – does the same thing:
yield return Timing.WaitUntilDone(_OtherCoroutine());
```

MEC Pro also supports waiting on a **CustomYieldInstruction**:

```
yield return Timing.WaitUntilDone(customYieldInstruction);
```

Extension Methods

Extension methods allow you to modify how a coroutine behaves while running it. They are very lightweight, so feel free to use them liberally or chain them together.

Delay

Delay runs the coroutine after a delay. This is far cleaner than passing in a delay parameter and putting a wait at the top of the function.

```
using UnityEngine;
using UnityEngine.UI;
using System.Collections.Generic;
using MEC;

public class ButtonExample : MonoBehaviour
{
    public Button button1;
    public Button button2;
    public Button button3;

    void Start ()
    {
        Timing.RunCoroutine(
            _MoveBackAndForth(button1).Delay(1f));
    }
}
```

```

        Timing.RunCoroutine(
            _MoveBackAndForth(button2).Delay(2f));
        Timing.RunCoroutine(
            _MoveBackAndForth(button3).Delay(3f));
    }

    private IEnumerator<float> _MoveBackAndForth(
        Button myButton)
    {
        Vector3 myPos = myButton.transform.localPosition;
        float time = 0f;
        while(time < 2f)
        {
            myPos.x += Timing.DeltaTime;
            time += Timing.DeltaTime;
            myButton.transform.localPosition = myPos;
            yield return 0f;
        }
        while (time < 4f)
        {
            myPos.x -= Timing.DeltaTime;
            time += Timing.DeltaTime;
            myButton.transform.localPosition = myPos;
            yield return 0f;
        }
    }
}

```

The code above will move three buttons to the right over 2 seconds and then back to the left. The interesting part is that during the `Timing.RunCoroutine` calls we used the `Delay` function to add a different delay to each call, so button1 will start moving after one second, button2 after 2 seconds, and button3 after 3 seconds.

Delay also has overloads that accept a condition function, which will hold the coroutine until the condition returns false:

```

// Delay until the function returns false
Timing.RunCoroutine(
    _DoWork().Delay(() => !isReady));

// Generic version to avoid closure allocations
Timing.RunCoroutine(
    _DoWork().Delay(someData, (data) => !data.IsReady));

```

DelayFrames

Similar to Delay, but waits for a specific number of frames instead of seconds:

```

Timing.RunCoroutine(_DoWork().DelayFrames(5));

```

CancelWith (Pro Overloads)

In addition to the GameObject-based CancelWith covered in the Quick Start Guide, MEC Pro adds two more overloads.

You can pass in a MonoBehaviour, and the coroutine will be cancelled when that script is destroyed or disabled:

```

Timing.RunCoroutine(
    _AnimateUI().CancelWith(this));

```

You can also pass in a function that returns a bool. The coroutine will be terminated as soon as that function returns false:

```
int framesLeft = 100;

void Start()
{
    Timing.RunCoroutine(
        _Coroutine().CancelWith(CancelFunction));
}

bool CancelFunction()
{
    if(framesLeft <= 0)
        return false;
    framesLeft--;
    return true;
}
```

If you want to be safe from more than three game objects going out of scope, you can chain calls to `CancelWith`:

```
Timing.RunCoroutine(
    _Coroutine()
        .CancelWith(obj1, obj2, obj3)
        .CancelWith(obj4));
```

Append and Prepend

These two functions can be used to chain two (or more) coroutines together. So this would turn right and then turn left:

```
Timing.RunCoroutine(_TurnRight().Append(_TurnLeft()));
```

They can also be used to append or prepend a delegate. So this might be used to run a coroutine and then destroy the object once it was done:

```
Timing.RunCoroutine(
    _MoveToFinalPosition(obj1)
        .Append(delegate { Destroy(obj1); }));
```

Of course in many cases you could have also just added the line to `Destroy(obj1)` to the end of the movement coroutine, but if you are using the same move function in several places and you don't normally want to destroy the object at the end then this can be a clean way to add that functionality without fracturing your code base.

There are also many cases where you might want to call some event once a coroutine has finished. If that event is related more to how you are calling the coroutine than what the coroutine is doing then it is better to encapsulate the code for the event in the place that is calling it, and this pattern does that.

Superimpose

This function superimposes two coroutines into a single handle. The combined coroutine won't finish until both of its contributing coroutines are done. This can be combined with the `WaitUntilDone` function to make a coroutine wait until both functions finish before continuing. Here's an example:

```
void Start()
{
    CoroutineHandle handle = Timing.RunCoroutine(
        _NetworkStream1()
```

```

        .Superimpose(_NetworkStream2()));
    Timing.RunCoroutine(_RunWhenDone(handle));
}

IEnumerator<float> _NetworkStream1()
{
    // Networking stuff happens here.
}

IEnumerator<float> _NetworkStream2()
{
    // Other networking stuff happens here.
}

IEnumerator<float> _RunWhenDone(CoroutineHandle handle)
{
    yield return Timing.WaitUntilDone(handle);
    // This runs as soon as both streams are done.
}

```

Hijack

This function alters the return value of a coroutine. This can be useful for cutscenes or replays where you want the same code to be executed but in slow motion.

```

float slowdown = 0.1f;
Timing.RunCoroutine(_MoveButton()).Hijack(input =>
{
    if(input <= Timing.LocalTime)
        input = (float)Timing.LocalTime;
    return input + slowdown;
}));

```

For the latest documentation, FAQ, or to contact the author visit trinary.tech.